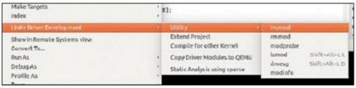

Figure 7: Driver development

shown in Figure 7. Or just click the *LinK+ Driver Utility* on *Toolbar*, or just right click on the *Editor*.

Short keys

To get the device driver utilities (*insmod, rmmod, modprobe* and *dmesg*) with short keys, press *Shift+Alt+L* and it will open the short keys window. Then press the first letter of the command, like 'd' for *dmesg*.

Extending a project

After creating the device driver project, if you want to add features, select the project under the *Project Explorer* view and click on *LinK+ Driver Utility* on *Toolbar*. Then, click on *Extend Project* to select the additional features you want to use. All features are optional. Finally, click on the *Finish* button.

Compiling for other kernels

As we know, Linux device drivers are version specific. If you want to compile the device driver project for other kernels, select the project in *Project Explorer* view and click on *LinK+ Driver Utility* on *Toolbar*. Click on *Compile For Other Kernel* and follow the steps indicated.

Static analysis using Sparse

If you want to check for static analysis, this IDE allows it through the Sparse tool. Just click on *LinK+ Driver Utility* on *Toolbar* and then click on *Static Analysis using Sparse*. It will open the LinK+ console, which shows Sparse warnings. Markers will also appear automatically on all warnings in the editor. Click on the warning icon in the editor and the LinK+ IDE will provide a quick fix option for code assistance and completion.

Eclipse local terminal plugin

The LinK+ IDE is also bundled with the Linux terminal plugin, which is provided by the Google ELT project plugins. To open the Linux terminal in the Eclipse IDE, click on *Window -> Show View -> Other -> General -> Terminal*.

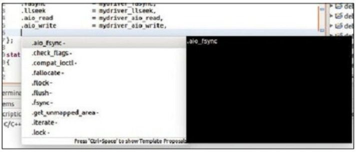

Figure 8: Function pointer structure

Code assistance and a quick-fix utility for LinK+ driver development

As we all know, development of Linux device drivers is quite cumbersome. We need to understand a lot of frameworks and data structures to build them. To ease this process, the LinK+ IDE has code assistance and code completion features.

To use this feature, press *Ctrl+Space* in the function pointer structure to get suggestions as shown in Figure 8. You can also type a few letters of the function name, and press *Ctrl+Space* to prompt for a list of names that start with a particular letter.

After writing the function pointer's name, the editor automatically gets an error message. In order to solve this, press *Ctrl+1* or click on the error marker icon for code suggestion. We can opt for any one of the suggestions for code completion.

Examples of Linux device driver programs

The LinK+ IDE is also bundled with sample programs for Linux device drivers. To view them, go to *Window->Show View->Other* and then go to *LinK+ Views*. Then, click on the device driver examples. You can also import the example program into the workspace as a project by right-clicking on the respective example program.

This article covers the basic features provided by the LinK+ Avatar release. I hope this information will help you to kickstart Linux kernel programming. In the next issue, I will discuss more features that include support for ARM architecture, network and block device driver development. END

By: Dileep K. Panjala

The author is a senior technical officer currently working with C-DAC, Hyderabad. This IDE has been developed by Dileep K. Panjala, A. Raghavedra Rao, Suman M., Devesh Gupta and S.V. Srikanth.